

Slide 1:

"Our high-level objective for this project was to take a custom Radix-2 FFT Butterfly unit, integrate it into the PULP SoC platform, and successfully drive it using a complete FFT software algorithm.

This is fundamentally a hardware/software co-design problem. Our shared goal was to validate the correct hardware/software interaction and prove that our custom silicon actually accelerates the math compared to a pure software approach.

To tackle this, we split the architecture down the middle. My focus—which is what I will be presenting today—is the Hardware Scope. This includes physically mapping the standalone butterfly accelerator to the system bus, building the translation wrapper, and ensuring the RTL datapath is mathematically perfect.

My colleague focused on the Software Scope: writing the C drivers, managing the memory pointers, and restructuring the software loop to push data to my hardware.

I am happy to report that we have successfully closed the loop. The hardware is no longer just a standalone unit test; it is now fully integrated and being successfully invoked by the complete FFT software application."

Slide 2:

"To understand how we integrated our butterfly accelerator, we first need to understand the chip it lives inside: the PULPissimo System-on-Chip.

If you think about a standard processor—like the one in your laptop—it uses hidden memory caches. Caches try to guess what data you need next. When they guess right, it's fast. When they guess wrong, the whole system freezes while it fetches the data. In high-speed telecommunications and 5G processing, a random system freeze is a disaster. You need absolute, predictable timing.

PULP solves this by throwing away the data cache completely. Instead, it uses a **TCDM**—which is essentially a giant, blazing-fast scratchpad. The golden rule of this architecture is 'No Surprises.' If you want hardware to work on data, the software must intentionally place it on the scratchpad first.

You can think of this chip as a highly organized factory floor.

- We have **The Manager** (the RISC-V core), who runs our C program but doesn't do the heavy lifting.
- We have **The Delivery Service** (the DMA), acting like forklifts that bring data in from the outside world.
- And finally, we have **The Heavy Machinery**—which is our custom FFT accelerator.

Our accelerator sits quietly on the factory floor until the manager brings it the exact numbers it needs and pushes the start button.

But, if the manager, the delivery forklifts, and our heavy machinery all try to grab things from the factory floor at the exact same millisecond, we would have a massive collision. And that brings us to how the chip manages its traffic..." (*Transition to Slide 2*)

Slide 3:

Speaker Notes: "So, if the manager and the delivery forklifts are all moving data around, how does the chip prevent traffic jams?

PULP solves this by building two completely separate highway systems inside the chip. The first is the **TCDM Interconnect**. You can think of this as the strict 'Fast Lane'. If a component is plugged in here, it must return data instantly in one clock cycle.

But our butterfly accelerator is doing heavy complex arithmetic, which takes several cycles. If we plugged it directly into the fast lane, the whole system would freeze waiting for the math to finish. Instead, we connect it to the **AXI Crossbar**. This is the 'Variable-Delay Highway'. AXI uses a handshaking system that essentially says: *'I received your data, but I need 7 clock cycles to process it. Safely pause my lane while I work.'*

Now, how does the RISC-V manager actually talk to our accelerator over this highway? First, we use **Address-Based Routing**. Hardware components don't have names; they just have zip codes. We carved out a chunk of the memory map starting at `0x1A40_0000` and told the AXI Crossbar: *'Route any traffic for this address straight to our FFT.'*

Once that road is paved, communication is split into two paths:

1. **The Control Path:** This is where the manager sends instructions. Instead of writing to normal RAM, the C-code writes configuration bits to our hardware's 'Memory-Mapped Registers'—like pressing buttons on the machine's control panel.
2. **The Data Path:** This is the flow of the actual heavy payloads—the complex numbers we want to multiply and add.

But there is one final hurdle. The AXI protocol is incredibly complex, and our custom Verilog code doesn't know how to speak it. We needed a translator..." (*Transition to Slide 3*)

Slide 4:

Speaker Notes: "We now have a paved AXI highway to our hardware. But there is a major problem: AXI is a highly complex, heavy protocol with five different channels just for routing traffic. Our 7-cycle FFT engine is a pure math machine—it just wants raw numbers.

So, we had to build a 'Protective Wrapper' that acts as a translator.

To build it, we wrote a blueprint file called a `.hjson`. This tells the system exactly what registers we need. Using a tool called `reggen`, this blueprint automatically generates our translation boundary. Inside this wrapper, a very specific **Protocol Downgrade** happens in two steps:

First, we downgrade **AXI to REG_BUS**. The wrapper catches the AXI transaction and strips away all the heavy routing overhead, leaving behind just a simple intermediate bus with an Address, the Data, and a Write-Enable signal.

Second, we downgrade **REG_BUS to Structs**. Auto-generated hardware cannot natively read floating bus interfaces. So, we use macros to pack those simple bus signals into strict data Structs—specifically, Request and Response structs.

Finally, those Structs are fed into our auto-generated Register File. The Register File cracks open those Structs and exposes three very simple, raw electrical wires to our custom Verilog:

1. First are the **.q wires**. These hold the operands perfectly stable so our hardware can read them at its own pace.
2. Second are the **.qe wires**. This is the spark. When the software writes to a trigger register, this wire pulses 'HIGH' for exactly one clock cycle.
3. Finally, the **.a wires**. Once our hardware finishes its math, it physically drives the answers onto these wires to be sent back up the chain.

Understanding these three wires—the stable data, the spark, and the result—is the key to how we optimized this entire accelerator..." (*Transition to the Optimizations*)

Slide 5:

Speaker Notes: "Now that we understand the goal, let's look at the exact math our heavy machinery is actually doing.

The core of any Fast Fourier Transform is a highly repetitive calculation called the Butterfly operation. Specifically, our accelerator implements a Radix-2 Decimation-in-Frequency, or DIF, butterfly.

This operation takes three complex numbers from the RISC-V processor: a Left sample, a Right sample, and a rotational coefficient known as a Twiddle Factor.

Inside the hardware, this splits into two distinct mathematical paths. The top path is the **Summation Path**. It simply takes the Left and Right complex samples and adds them together. The bottom path is the **Difference and Rotation Path**. It first subtracts the Right sample from the Left, and then performs a heavy complex multiplication against the Twiddle factor.

In a 4096-point FFT, this exact set of equations has to be calculated thousands of times. By offloading this specific block of math into custom silicon, we free up massive amounts of processor time. But to do that efficiently, we had to carefully design the internal RTL architecture..." (*Transition to Slide 7*)

Slide 6:

"Moving inside the accelerator, here is the physical architecture of our RTL datapath.

The entire design is heavily parameterized. By simply changing the `IWIDTH`, `CWIDTH`, and `SHIFT` values at synthesis, the same Verilog code can adapt to different fixed-point Q-notations.

At the core, the input data immediately hits our Add and Subtract units. But the most critical engineering choice we made resides in the Complex Rotation block. Originally, we considered using highly pipelined multiplier IPs—specifically modules known as `bimpy` and `longbimpy`. However, we chose to strip those out and use standard multiplication logic. Why? Because the standard C-code we are benchmarking against does not use pipelined multiplication. To confidently state that our hardware accelerates the algorithm, we needed a pure, apples-to-apples comparison.

Finally, we included two essential supporting mechanisms. First is a `convround` module that performs Banker's Rounding to maintain signal integrity. Second is a pipeline alignment stage, which guarantees that the fast addition path and the slower multiplication path arrive at the output registers on the exact same clock cycle."

Slide 7:

Speaker Notes:

"Once we established our baseline datapath, we looked for ways to make it more efficient.

In digital design, multipliers are the most expensive structures you can put on a chip. They eat up massive amounts of silicon area and DSP slices. A standard complex multiplication—multiplying the data by the Twiddle factor—requires four separate real multipliers.

To optimize this, we applied the Karatsuba algorithm. By doing a little bit of mathematical reorganization, we calculate three intermediate products instead of four.

Of course, in engineering, there is always a trade-off. The 'price' we pay for this optimization is that we have to add a few extra Adders and Subtractors before and after the multipliers to reconstruct the final Real and Imaginary parts.

However, an adder is incredibly cheap in hardware compared to a multiplier. Trading one massive multiplier for a handful of tiny adders is a massive net win, drastically reducing our overall hardware footprint and making the accelerator highly area-efficient."

Slide 8:

Speaker Notes: "Because our hardware is a pipeline, it suffers from a classic digital design problem: different mathematical operations take different amounts of time.

The Summation path is just an addition, which is very fast. But the Difference path has to travel through that entire Karatsuba multiplication block, which takes several clock cycles. If we just let the data flow naturally, the upper half of our butterfly would finish long before the lower half.

To solve this latency mismatch, we built what is essentially a 'Shadow Pipeline.'

When the software triggers the hardware, it fires a signal called `i_aux`. You can think of this as dropping a physical token onto a parallel conveyor belt. As the complex numbers move cycle-by-cycle through the math stages, that token shifts right alongside them.

This does two things. First, it forces the fast addition path to wait, guaranteeing that both halves of our complex answer pop out perfectly aligned at the exit door. Second, when that token finally reaches the end of its conveyor belt, it pops out as our `o_aux` signal. This gives the rest of the system a perfectly deterministic flag that says, *'The pipeline is finished, and the answers are ready to be read.'*

Slide 9:

Speaker Notes:

"So, at this point in the design, we had a mathematically perfect, 7-cycle FFT engine. But a great engine is useless if it's not actually plugged into the car. We needed to integrate it into the PULPissimo SoC.

Our objective was to take this raw Verilog and expose it to the RISC-V processor as a fully memory-mapped AXI peripheral. To do this, we built an Integration Stack.

As I mentioned in the introduction, we started with a Blueprint—an `.hjson` file. We fed this into the `reggen` tool to build our Register File. We then wrapped both our custom RTL and that Register File inside an AXI Wrapper.

Once we had that packaged module, we had to physically wire it into the main chip. We opened up the `pulp_soc.sv` top-level file and instantiated our wrapper. We extended the AXI interconnect to add a new slave port, and finally, we updated the global address map. By inserting a new AXI decoding rule, we guaranteed that any time the C-code asked for our specific memory address, the traffic would be perfectly routed to our accelerator."

Slide 10:

Speaker Notes:

"If we zoom into the wrapper we just placed on the SoC, here is exactly how data flows from the software to our hardware.

Looking at the block diagram, traffic arrives from the PULP processor on the left via the AXI Master Interface. It enters the top half of our wrapper: the AXI-to-Register Converter. As we

discussed earlier, this is our 'mailroom.' It strips away the heavy AXI protocol and converts it into a simple Register Bus.

That simple bus feeds directly into the bottom half: our Autogenerated Register File. This is the actual memory bank where our operands and twiddle factors are physically stored.

Finally, we hit the boundary interfaces: `reg_file_to_ip` and `ip_to_reg_file`. This is where the register file unpacks the data into those raw, stable `.q` wires to feed the accelerator, and reads the answers back from the `.d` wires.

By structuring the wrapper this way, our 7-cycle datapath never has to know that the AXI bus even exists. It just does the math."

Slide 11:

"Once our hardware wrapper was verified, the next critical phase of our co-design was physically wiring it into the PULP system. We had to perform some top-level surgery on the SoC.

The key modifications happened inside the `pulp_soc.sv` file. First, we instantiated our custom wrapper alongside the existing peripherals. To connect it, we had to add a brand-new AXI slave port to the main crossbar and extend the interconnect logic so the traffic could physically reach our module.

The most important software-facing modification was the Address Allocation. We updated the global address map to assign a dedicated, permanent memory block to our accelerator. You can think of this as giving our hardware a specific 'zip code' inside the chip. Because it now has a memory-mapped region, my software colleague can interact with my hardware using nothing but standard C pointers and basic load/store instructions.

The final result of these modifications is that our butterfly unit is no longer an isolated testbench module. It is a fully visible, natively accessible component on the main system bus, ready to accept AXI transactions from the processor."

Slide 11:

Speaker Notes: "Once our hardware wrapper was verified, the next critical phase of our co-design was physically wiring it into the PULP system. We had to perform some top-level surgery on the SoC.

The key modifications happened inside the `pulp_soc.sv` file. First, we instantiated our custom wrapper alongside the existing peripherals. To connect it, we had to add a brand-new AXI slave port to the main crossbar and extend the interconnect logic so the traffic could physically reach our module.

The most important software-facing modification was the Address Allocation. We updated the global address map to assign a dedicated, permanent memory block to our accelerator. You can

think of this as giving our hardware a specific 'zip code' inside the chip. Because it now has a memory-mapped region, my software colleague can interact with my hardware using nothing but standard C pointers and basic load/store instructions.

The final result of these modifications is that our butterfly unit is no longer an isolated testbench module. It is a fully visible, natively accessible component on the main system bus, ready to accept AXI transactions from the processor."

Slide 12:

Speaker Script for "Software & Verification (1)"

Speaker Notes: "With the hardware successfully wrapped and integrated into the PULP SoC, we moved on to verification. Step 1 was proving mathematical correctness—we had to guarantee that the hardware and software were speaking the exact same language.

To do this, we developed a bare-metal C test application. The execution flow directly utilizes the driver functions we discussed: the software writes the operands and coefficients, the hardware is triggered, the software polls the status, and finally, it reads back the results.

If you direct your attention to the simulation log on the right side of the slide, you can actually see this HW/SW co-design in action. This log beautifully interleaves software `printf` statements from the RISC-V core with hardware `$display` debug prints coming directly from our Verilog.

Look at Test Case 1, for example. The software loads the Twiddle factor and the complex inputs. Instantly, you see the hardware print that the `i_aux` AXI trigger was received. Exactly 7 cycles later, the hardware pipeline finishes, asserting `o_aux`. The software then immediately breaks its polling loop, pulses its own success flag, and reads out the calculated Sum and Difference.

We ran this across multiple test cases covering various Twiddle factors and edge cases. The observed results were perfect. The butterfly arithmetic perfectly matched our golden standalone testbenches, the trigger signaling functioned exactly as designed, and we achieved fully successful, end-to-end communication between the RISC-V processor and our custom silicon."

Slide 13:

Speaker Notes: "Once we had the basic wrapper working, we looked for ways to optimize the communication between the RISC-V processor and our hardware. The first major improvement we made was implementing an Auto-Trigger mechanism.

Normally, to drive an accelerator, the software has to perform three distinct steps over the bus: write the first operand, write the second operand, and finally, do a third write to a dedicated 'Start' register to tell the hardware to begin. Across a 4096-point FFT, that third write adds up to thousands of wasted clock cycles on the AXI bus.

We decided to eliminate that third write entirely.

When we built our register blueprint, we configured the Right Operand to generate a Write-Enable pulse—the `.qe` wire. In our wrapper Verilog, we took that `.qe` wire and directly plugged it into our accelerator's `i_aux` trigger port.

Because of this simple wire connection, the software only has to write the Left Operand and then the Right Operand. The very act of the data arriving physically sparks the hardware into action on that exact clock edge. It immediately drops the data into Stage 1 of the pipeline, completely eliminating the need for a separate 'Start' command over the AXI bus."

Slide 14:

Gemini said

Here is the draft for **Optimization 2: Auto-Status Management & Write-Back**.

This slide tackles the clever hardware logic you wrote at the very bottom of your `simple_butterfly_top` module. It shows the reviewers that you didn't just design a math engine; you engineered a system that actively prevents software bugs.

Slide 18: Optimization 2 – Auto-Status Management

[Slide Content]

- **Title:** Optimization 2: Auto-Status & Secure Write-Back
- **The Polling Problem:**
 - Software must poll a "Status" register to know when the 7-cycle computation is done.
 - Traditionally, software must manually write a '0' to clear this flag before starting a new calculation, wasting another AXI transaction.
- **Our RTL Solution (The `.de` Wire):**
 - We used the Data Enable (`.de`) wire to make the hardware self-managing.
 - **Auto-Clear:** assign `status.de = w_o_aux | op_right.qe`;
 - The exact moment a new trigger fires (`op_right.qe`), the hardware forces the status bit back to '0'.
 - **Auto-Set:** When the 7-cycle pipeline finishes (`w_o_aux`), it drives the status bit to '1'.
- **Secure Data Latching:**
 - The output results (`result_left`, `result_right`) are strictly gated by `.de = w_o_aux`. The register file physically cannot capture garbage data mid-calculation; it only latches the final, mathematically valid answers.

[Detailed Explanation / Speaker Script]

Speaker Notes: "Our second major wrapper optimization focused on the back-end of the transaction: how the software reads the answers.

Because our accelerator takes 7 cycles, the RISC-V processor has to poll a Status register to know when the data is ready. In a naive design, after reading the answers, the software would have to send another AXI write command to manually clear that Status flag back to zero before starting the next butterfly.

We completely eliminated this requirement using the Data Enable, or .de, wires exposed by our autogenerated register file.

We wrote a simple logical OR gate in our Verilog. The exact moment the processor writes the Right Operand—triggering the start of a new computation—our hardware automatically captures a '0' into the Status register. It Auto-Clears. Then, 7 cycles later, when the pipeline done-flag (w_o_aux) pops out of our shift register, it automatically writes a '1'.

Furthermore, we tied that same done-flag to the Data Enable ports of our Result registers. This provides hardware-level security. It physically prevents the register file from accidentally capturing intermediate garbage data while the math is running. The processor is guaranteed to only ever read perfectly valid, cycle-accurate outputs.

Slide 15:

Speaker Notes: "In Step 1, we proved the math was flawless. In Step 2, our focus shifted entirely to performance and latency profiling. It wasn't enough for the accelerator to just work; it had to be fast.

We benchmarked this in three distinct phases. In our initial Phase 1 baseline, we wrote a naive driver. We were passing the entire complex Twiddle factor over the AXI bus for every calculation and using standard `longbimpy` multiplier IPs in the hardware. The hardware took about 14 to 16 cycles, but because of bus overhead, the actual software round-trip latency was a massive 442 clock cycles per butterfly.

For Phase 2, we cleaned up the software driver overhead but kept the architecture the same. This brought our software latency down to a much better 59 cycles, but the AXI bus was still a bottleneck.

Finally, in Phase 3, we implemented the architectural upgrades I presented earlier. We stripped out `longbimpy` for standard multipliers to achieve a locked 7-cycle hardware datapath. Instead of sending heavy Twiddle factors over the bus, we sent a lightweight Twiddle *index* to trigger the internal ROM. Combined with our Zero-Overhead Auto-Trigger, we slashed the total software latency down to just **24 cycles**.

We took this from 442 cycles down to 24 cycles—proving our hardware/software co-design was a massive success."

Questions Slide:

1. Software Instruction Overhead (The Processor)

Before data ever hits the bus, the RISC-V processor has to physically move it around using assembly instructions.

- **Memory Fetches (Load Instructions):** The CPU must first execute `lw` (Load Word) instructions to pull the complex numbers out of main memory (TCDM/L2) and put them into its internal CPU registers.
- **Register Writes (Store Instructions):** The CPU then executes `sw` (Store Word) instructions to push those numbers out of its internal registers and toward your hardware's memory-mapped addresses.
- **Pointer Arithmetic:** If this is inside an FFT loop, the CPU spends 1-2 cycles updating the memory pointers to fetch the *next* set of data.

2. Interconnect & Protocol Overhead (The AXI Bus)

The AXI crossbar is a shared highway, and getting onto it requires a strict handshake protocol.

- **Address Decoding & Routing:** When the CPU issues a write, the crossbar takes 1-2 cycles just to look at the address, decode it, and figure out which slave port (your wrapper) to route it to.
- **AXI Handshaking:** Every single read or write requires a physical handshake. For a write, the bus must assert `AWVALID/AWREADY` (address phase), `WVALID/WREADY` (data phase), and wait for a `BVALID/BREADY` (response phase). Even with zero wait-states, this consumes cycles.
- **Bus Contention (Arbitration):** If the DMA or another core is using the crossbar at the exact same moment, your CPU's transaction has to wait in line.

3. Translation & Wrapper Overhead (The Interface)

Once the AXI transaction reaches your `simple_butterfly_top` module, it doesn't instantly hit your 7-cycle datapath.

- **Protocol Downgrade:** Your `axi_to_reg_intf` module has to catch the AXI signals and translate them into the simpler `REG_BUS` format. This sequential logic usually costs 1 clock cycle.
- **Register Latching:** The autogenerated register file takes 1 clock edge to physically latch the data into the `.q` flip-flops before your hardware can "see" it.

4. Synchronization Overhead (The Polling Loop)

Your software has to know when the 7 cycles are finished.

- **AXI Read Latency:** To check the Status register, the CPU issues a read. This transaction must travel all the way through the AXI bus, through your wrapper, read the `status.q` bit, and travel all the way back to the CPU.
- **Branching Penalty:** The CPU is executing a `while(status == 0)` loop. When the status finally flips to '1', the CPU breaks the loop, which can cause a 1-2 cycle pipeline flush (branch misprediction penalty) inside the RISC-V core.

5. Data Read-Back Overhead (Extracting the Results)

Once the CPU knows the data is ready, it has to pull the answers out.

- **Two More AXI Reads:** The CPU must issue two distinct AXI read transactions to fetch `result_left` and `result_right`.
- **Storing to Memory:** Finally, the CPU must execute `sw` (Store) instructions to write those finished answers back into the main TCDM memory block so the next stage of the FFT can use them.

You hit the nail on the head. The 7-cycle difference between your 24 cycles and your colleague's 17 cycles comes down to a combination of **how the driver is written** and **how the specific environment (SDK vs. bare Runtime) compiles that C code**.

Getting the total latency down to 17 cycles (which is only **10 cycles of overhead** around your 7-cycle hardware) means your colleague is squeezing every drop of efficiency out of the RISC-V core.

Here is exactly where those 7 extra cycles are hiding in your version, and why the SDK/driver approach changes the math.

1. Driver Implementation: Function Calls vs. Macros/Inline

If you wrote your driver as a standard C function—for example, `void run_butterfly(int left, int right)`—the RISC-V processor has to execute a "jump" instruction to find the function, push variables to the stack, execute the code, and then "return" to the main loop. That setup and teardown easily costs 5 to 8 clock cycles.

- **The SDK/Colleague Approach:** The PULP SDK relies heavily on Hardware Abstraction Layer (HAL) macros and `static inline` functions. When the code is compiled, the compiler literally copies and pastes the raw assembly instructions directly into the main loop. There is zero function-call overhead.

2. Compiler Flags and PULP Custom Instructions

The PULPissimo core (RI5CY/CV32E40P) isn't just a standard RISC-V processor; it has custom "Xpulp" DSP extensions.

- **The Runtime Approach:** A generic runtime build might compile standard RV32IMC assembly. It uses standard `lw` (load) and `sw` (store) instructions, and manually updates memory pointers.
- **The SDK/Colleague Approach:** The SDK build system aggressively targets the custom PULP extensions. It will use **post-incrementing load/stores** (e.g., `p.lw`). This allows the processor to load a variable and update the memory pointer in a single, 1-cycle instruction. Over the course of reading inputs and writing outputs, this easily saves 3 to 4 cycles.

3. The Polling Loop (Branch Penalties)

How you wait for the hardware matters immensely at the cycle level.

- **Standard Polling (Your Driver):** If you use a simple `while (REG->STATUS == 0);`, the processor is constantly reading the bus, evaluating a condition, and jumping back to the top of the loop. When the status finally changes, the processor suffers a 1-to-2 cycle "branch misprediction penalty" as it breaks out of the loop.
- **Optimized Polling:** Highly tuned drivers will sometimes unroll this, or if the SDK is utilized fully, it might use the PULP Event Unit to put the core to sleep for exactly 7 cycles (using a `wfi` instruction) and wake up exactly when the `o_aux` flag triggers an event, eliminating bus polling entirely.

4. Memory Locality (L1 vs. L2)

- **Instruction Fetching:** If the SDK script naturally maps your colleague's driver code directly into the tightest, fastest L1 Instruction Cache (or TCDM), the processor never stalls. If a basic runtime script maps your driver into a slightly slower memory region, the core might spend 1 or 2 cycles just fetching the next instruction.